AQ-06 · attestqa

Local Test Strategy and Target Applications

Version 1.4 (for sign-off) · 2026-10-10 · Author: Vijay Javvadi

1. Two different kinds of testing

This document covers two things that are easy to mix up. Section 2 is how we test attestqa itself, so we can trust it. Sections 3 to 6 are how attestqa is used to test other people's websites on our machine, which is both the product working and the data for the papers.

2. Testing attestqa itself

Level	What	Tool	Runs
Unit	Gate rules, triage, rerun predicate, config validation, scrubber	vitest	Every commit, Windows
Rule regression	One test per refusal rule, named after the run that produced it (the existing pattern: `executepath.test.ts`, `duplicate-bug.test.ts`, ...)	vitest	Every commit
Provider conformance	Same suite against every provider with a fake transport; nightly against real endpoints where keys exist	vitest	Commit (fake), nightly (real)
Ledger replay	Recorded ledgers fed through a model that replays recorded decisions; the report must match the recorded report exactly. Includes the 1.x corpus	vitest + fixture ledgers	Every commit
UI components	Each screen against `FakeClient` replaying a ledger; axe on every screen	Playwright component tests	Every commit
Desktop end-to-end	Launch the packaged app with Playwright's Electron support; first run, Attest on Juice Shop, approval dialog, crash-recovery (kill the worker mid-run and check `run_aborted`)	Playwright `_electron`	Before every release
Type adapters	`api` scope enforcement (requests to hosts outside `api_hosts` refused); `sql` parser rejects every write and DDL form in a fixed corpus of statements; `zap.active` absent from the tool list unless authorisation is recorded	vitest	Every commit
Tracker adapters	ADO and Jira adapters against recorded API responses; `createBug` impossible without an approval event; no update method exists	vitest	Every commit

Level	What	Tool	Runs
Security checks	Renderer has no Node access; no secret appears in renderer bundle, IPC log, ledger, or app log; CSP present	scripted checks	Before every release
Install test	Fresh Windows 11 VM, SmartScreen default, standard user account	manual checklist	Before every release

The rule that matters most: a change to core is not accepted unless ledger replay still produces the same reports for the recorded runs, or a decision record explains why it should not.

3. Local environment

Item	Version / note
OS	Windows 11 (the development laptop)
Node	24 LTS line (matches the desktop app's bundled runtime)
Docker Desktop	For locally hosted targets (Juice Shop)
Playwright	Pinned per target project in `package.json`
Model accounts	Anthropic key (existing); a second provider for the cross-model study (OpenAI or Azure OpenAI) — decision D8
ai-governance sidecar	`http://127.0.0.1:4900`, as used in the Juice Shop runs
Secrets	`.env` files, git-ignored, per target; app credentials `AQA_APP_USER` / `AQA_APP_PASS`

4. Target applications

We use websites only, never their source code. A site qualifies if its owner presents it as a place to practise testing or automation, or if we run it locally from a public image. Every target is recorded in `TestRunApplication/targets.yaml` with its URL, owner, what kind of application it is, why we believe we may automate against it, the date that was checked, and any rules the owner states.

4.1 Tier L — run locally (Docker)

Target	Kind	Why it is useful	Notes
OWASP Juice Shop (`localhost:3000`)	E-commerce with intentional vulnerabilities	Continuity with Paper I (story AB#44 and the existing defects); realistic SPA; we control resets	Only target where security-flavoured checks are allowed, because it runs on our machine

4.2 Tier H — hosted practice applications

These are listed on testing-practice directories (see Sources). The URLs marked "verify" were named in those directories but not opened during writing; the first month of work

(before v1.0.0) confirms each one and records what its owner says about automation.

Target	URL	Kind	Why it is useful
Swag Labs (Sauce Labs)	https://www.saucedemo.com	E-commerce	Demo credentials provided; built-in "problem" and "glitch" users give known misbehaviour to detect
Demoblaze	https://www.demoblaze.com	E-commerce	Modals, alerts, cart; registration optional
ParaBank (Parasoft)	https://parabank.parasoft.com/parabank/index.htm	Online banking	Registration, transfers, forms with validation
The Internet	https://the-internet.herokuapp.com	Catalogue of UI controls	Dynamic loading, frames, auth dialogs; good for locator-heal checks. Never upload private files: uploads are public
UI Testing Playground	http://uitestingplayground.com	Hard UI cases	Dynamic ids, delays, hidden layers; direct test of the heal role
Formy	https://formy-project.herokuapp.com	Forms	Simple, stable; good first Explore target
CandyMapper	https://candymapper.com	Site with deliberate errors	Known faults give ground truth for defect finding
Testing Toolshop	practicesoftwaretesting.com (verify)	E-commerce with testing hints	Realistic catalogue, filters, checkout
Restful Booker Platform	automationintesting.online (verify)	Hotel booking with admin	UI plus API; resets on a schedule
Playwright TodoMVC demo	demo.playwright.dev/todomvc (verify)	Single-page todo app	Small, deterministic baseline for the cross-model study

Sites with deliberate faults (Swag Labs' problem user, CandyMapper, and any site whose owner publishes a list of known bugs) are especially valuable: they let us measure whether attestqa finds real defects and whether it reports them honestly, against an answer key we did not write.

4.3 API targets (v2.0–v2.1)

Target	Where	Why
Juice Shop REST API	`localhost:3000/rest`, `/api`	Same stories as the web tests; the protected-route defect (#83) should reproduce one layer down
Restful Booker API	the API behind the Restful Booker platform (verify)	Published practice API with auth and CRUD
Swagger Petstore	the public Petstore sample (verify)	Comes with an OpenAPI file, so the contract oracle can be tested directly

4.4 Security targets (v4.0–v4.1)

Local Juice Shop only, for both passive checks and active scanning. Hosted practice sites get passive checks at most (headers, cookie flags, TLS), at human pace, and only where their owner's statements allow automated testing; active scanning of any hosted site is not offered by the tool (AQ-04 §2.6).

4.5 ETL sources (v5.0)

There are no public ETL "websites", so the ETL targets are built locally from data we already have: the synthetic banking, e-commerce, healthcare and HR databases and schemas in `E:\Sy`. A small reference load (for example orders from `ecommerce.db` into a reporting schema) is written with a mapping document, and transformation faults are seeded into copies of it. The list of seeded faults is written down and hashed before any run, so it serves as an answer key nobody can adjust afterwards.

4.6 Not used

Real production websites or databases, anything requiring a real person's data, and any site whose owner forbids automated access. Active security scanning, load testing and fuzzing are only done against Tier L.

5. Rules of engagement for hosted targets

1. Browser automation at human pace: one worker, no parallel shards, a minimum delay between navigations set in each project's config.
2. Test accounts only. No real names, emails or payment details; generated values are recorded in the project's seed file.
3. Stop on HTTP 429 or 5xx bursts; the run is marked environmental and not retried the same day.
4. No attempts to break authentication, inject payloads or reach admin functions not offered to ordinary users.
5. Uploads: only generated, harmless files, and only where the site invites uploads.
6. The permission basis and the date it was checked are re-verified before each study begins, and recorded in `targets.yaml`.
7. If an owner asks us to stop, we stop, and the target is marked withdrawn with the date.

6. How a target is onboarded and tested

1. **Register.** Add to `targets.yaml`, check the owner's statements, record the date.
2. **Write stories first.** Three to eight stories per site, in `stories/*.md` or as work items in a test ADO or Jira project, written from the site as a user would see it, before attestqa explores it. Their hashes go into `research/runs-manifest.csv`. This stops anyone (including us) tuning stories to results.
3. **Explore.** One Explore run; drafts and open questions reviewed by Vijay.
4. **Specify.** Each story turned into approved tests.
5. **Attest three times** on separate days, so flakiness and site resets are visible rather than mistaken for defects.
6. **Record.** Each run's id, target, story, mode, provider, model, core commit and counts go into the manifest; anything new goes into `findings.jsonl` (AQ-08).

7. What counts as success for a target

The generated suite runs green or red for reasons a person agrees with; every failure has a triage verdict with evidence; confirmed defects match known faults where an answer key exists; every attempted weakening is refused and logged; and the project runs with `npx playwright test` without attestqa.

Sources

- Evil Tester, "Other Testing Practice Sites": https://testpages.eviltester.com/reference/other-sites
- BugBug, "Best Selenium Practice Websites in 2026": https://bugbug.io/blog/testing-frameworks/best-selenium-practice-websites/